

FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO
DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE

Criação de Camarão em Cativeiro com Monitoramento IoT para Gastronomia

Davi Mathais de Almeida
Leandro Augusto de Muniz
Tiago de Lara Rodrigues

Índice

1	CANVAS: Modelo de Negócio e Estratégia	3
2	SWOT: Análise Estratégica	3
2.1	Recomendações Estratégicas	4
3	UX/UI: Design de Interface e Experiência do Usuário	4
3.1	Guia de Estilos	5
4	Website: Estrutura e Especificações Técnicas Web	5
4.1	Mapa do Site da Equipe	5
4.2	Conteúdo das Páginas	5
4.3	Screenshots do Website da Equipe	6
4.4	Especificações Técnicas do Site da Equipe	8
5	Diagramas UML: Modelagem Visual do Sistema	8
5.1	Diagrama de Casos de Uso	9
5.2	Diagrama de Classes	9
5.3	Diagrama de Sequência	11
5.4	Diagrama de Atividades	11
5.5	Diagrama de Componentes	12
5.6	Diagrama de Implantação	13
5.7	Diagrama de Estados	13
6	DevOps: Infraestrutura, Automação e Entrega Contínua	15
6.1	Pipeline CI/CD	15
6.2	Configuração CI/CD	15
6.3	Infraestrutura como Código	16
6.4	Infraestrutura do Sistema	16
6.4.1	Arquitetura de Rede	17
6.4.2	Diagrama de Infraestrutura	17
6.5	Arquitetura de Deploy	18
6.6	Configuração Docker Compose (Desenvolvimento)	19
6.7	Monitoramento e Alertas	19
6.8	Métricas e SLA	20
	Referências	22

Lista de Artefatos e suas Finalidades

- **CANVAS:** Modelo de negócio e estratégia
 - **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
 - **UX/UI:** Design de interface e experiência do usuário
 - **Website:** Estrutura e especificações técnicas web
 - **Diagramas UML:** Modelagem visual do sistema
 - **DevOps:** Infraestrutura, automação e práticas de entrega contínua
-

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas (OSTERWALDER; PIGNEUR, 2011) é uma ferramenta visual estratégica¹ que permite descrever, analisar e projetar modelos de negócio de forma clara e objetiva. O canvas é dividido em nove blocos fundamentais que cobrem as principais áreas de um negócio.

Tabela 1: Business Model Canvas – Visão Geral

Parcerias Principais	Atividades Chave	Proposta de Valor	Relacionamento	Segmentos
Consultores da carcinicultura, produtores de camarão	Interpretação de dados sobre temperatura, pH e amônia, monitoramento comunicando com a plataforma	Automatização no cuidado dos cativeiros, redução de custos, aplicação juntamente com sistema IoT	Suporte técnico, treinamentos, chat direto com a empresa	Carcinicultura, aquicultura, sistema de monitoramento
	Recursos Principais		Canais	
	[Sensores de temperatura, pH e amônia, microcontrolador ESP32, dispensador automatizado de ração]		[Comunicação via redes sociais, e-mail e telefone]	
Estrutura de Custos		Fontes de Receita		
Desenvolvimento do sistema, equipamentos IoT, marketing		Plano de assinatura mensal e anual, treinamentos, instalação do sistema		

2 SWOT: Análise Estratégica

A Análise SWOT (Strengths, Weaknesses, Opportunities, Threats) ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) (CHIAVENATO; SAPIRO, 2020) é uma ferramenta fundamental para planejamento estratégico² que permite identificar fatores internos e externos que impactam o projeto. Analisa forças, fraquezas, oportunidades e ameaças do projeto.

¹O Canvas foi desenvolvido por Alexander Osterwalder e é amplamente utilizado em startups e empresas estabelecidas para modelagem de negócios.

²A análise SWOT foi criada na década de 1960 por Albert Humphrey no Stanford Research Institute.

Tabela 2: Análise SWOT do Projeto

FATORES INTERNOS	
FORÇAS (Strengths)	FRAQUEZAS (Weaknesses)
• Integração entre aplicação e sensores IoT em tempo real • Automação de alimentação • Alertas imediatos e gráficos simples • Interface intuitiva e responsiva • Redução de custos operacionais	• Dependência de internet • Necessidade de manutenção dos sensores • Necessidade de treinamento dos usuários • Custo inicial de implementação
FATORES EXTERNOS	
OPORTUNIDADES (Opportunities)	AMEAÇAS (Threats)
• Crescimento do mercado de aquicultura • Parcerias com instituições de pesquisa • Expansão para outros tipos de monitoramento ambiental • Aumento da demanda por soluções sustentáveis	• Concorrência de soluções similares • Resistência à adoção de novas tecnologias • Problemas de infraestrutura em áreas rurais • Risco de falhas técnicas • Questões regulatórias e de conformidade

2.1 Recomendações Estratégicas

Com base na análise SWOT, as principais estratégias devem focar em:

- **Forças + Oportunidades:** Investir no marketing digital para aproveitar o crescimento do mercado
- **Forças + Ameaças:** Reforçar a segurança para se diferenciar da concorrência
- **Fraquezas + Oportunidades:** Buscar parcerias para ampliar a presença no mercado
- **Fraquezas + Ameaças:** Criar planos de contingência e melhorar a documentação

3 UX/UI: Design de Interface e Experiência do Usuário

UX (User Experience) e UI (User Interface) (ROGERS; SHARP; PREECE, 2013) são disciplinas complementares focadas em criar produtos digitais que sejam funcionais, acessíveis e agradáveis de usar.³ O UX concentra-se na experiência geral do usuário, enquanto o UI cuida dos elementos visuais e interativos.

³Ferramentas populares para design UX/UI incluem Figma, Adobe XD, Sketch e InVision.

3.1 Guia de Estilos

Tabela 3: Paleta de Cores do Projeto

Elemento	Cor	Código HEX	Uso
Primária		#aed5ff → #ffc2c2	Botões principais, destaques
Secundária		#2261A6	Elementos secundários
Fundo		#FFFFFF	Background padrão
Fundo Informações		#F8fafc	Background de informações
Texto		#000000	Texto principal
Sucesso		#166534	Mensagens de sucesso
Erro		#991b1b	Mensagens de erro

4 Website: Estrutura e Especificações Técnicas Web

Esta seção documenta o website institucional/portfólio da equipe que desenvolveu a aplicação. O site da equipe serve para apresentar os membros, projetos realizados, competências e informações de contato.

4.1 Mapa do Site da Equipe

O mapa do site (sitemap) mostra a hierarquia e organização das páginas do website da equipe:

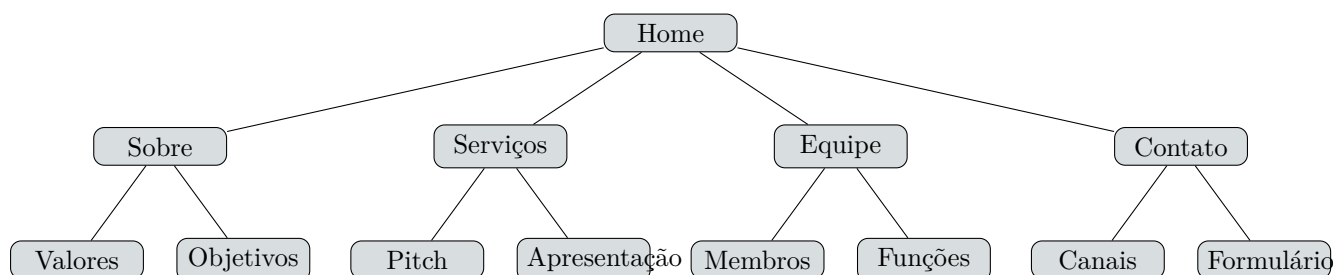


Figura 1: Estrutura de navegação do website da equipe

4.2 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Home:** Apresentação inicial da equipe
- **Sobre Nós:** Conteúdo sobre nossos valores, missão e visão
- **Serviços:** Descrição dos serviços oferecidos pela equipe
- **Equipe:** Perfil dos membros (nome, foto, função, LinkedIn/GitHub)
- **Contato:** Formulário de contato e links para redes sociais

4.3 Screenshots do Website da Equipe



Figura 2: Página inicial do website da equipe



Figura 3: Página "Sobre Nós" do website da equipe

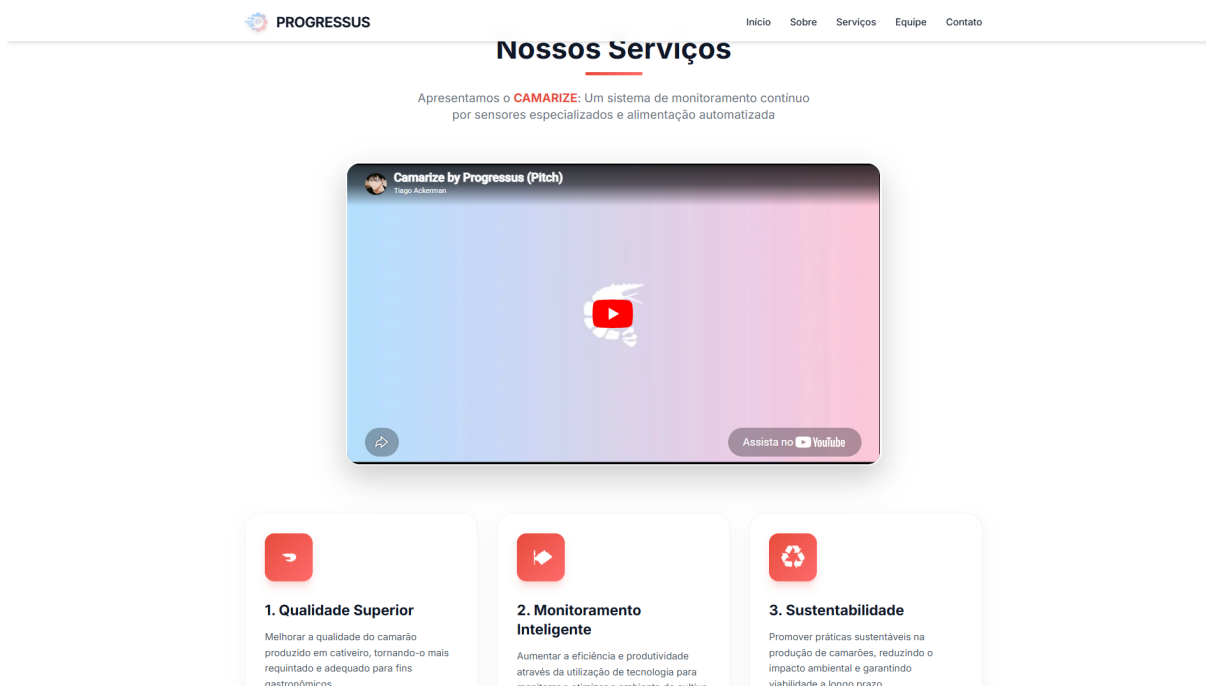


Figura 4: Página "Serviços" do website da equipe

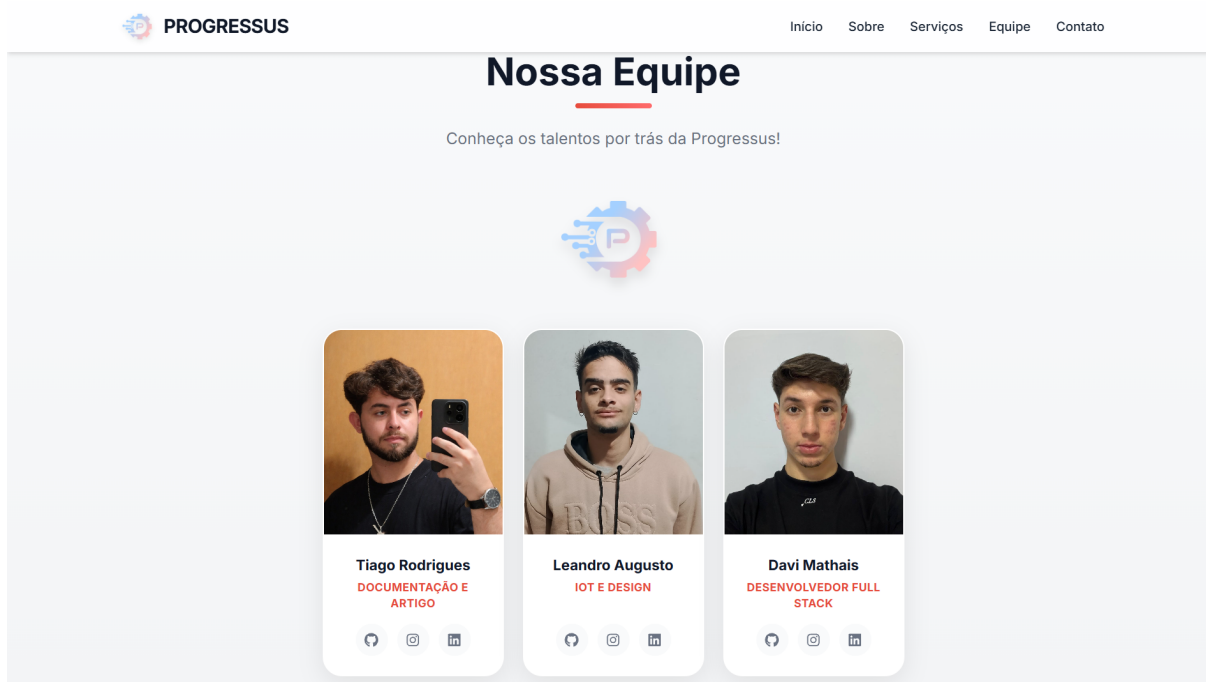


Figura 5: Página "Equipe" do website da equipe

 **PROGRESSUS**

[Início](#)
[Sobre](#)
[Serviços](#)
[Equipe](#)
[Contato](#)

Entre em Contato

Informações de Contato

 Email
contato@camarize.com

 Localização
Registro, SP

 Telefone
+55 (13) 99656-6700

Siga-nos nas redes sociais



Envie uma mensagem

Nome Completo

E-mail

Mensagem

Enviar Mensagem

Figura 6: Página "Contato" do website da equipe

4.4 Especificações Técnicas do Site da Equipe

Tabela 4: Stack Tecnológica do Website da Equipe

Camada	Tecnologia	Descrição
Frontend	React 18	Biblioteca JavaScript para interfaces
Build Tool e Bundler	Vite	Ferramenta de build de altíssima performance e dev server
Linguagem	TypeScript	Superset do JavaScript que garante tipagem estática segura
Estilização e Design	Tailwind CSS v4	Framework CSS utilitário para responsividade e temas
Motor de Animações	Framer Motion	Biblioteca declarativa para animações e transições complexas
Iconografia Vetorial	Lucide / React Icons	Bibliotecas de ícones modernos, responsivos e otimizados
Serviço de Mensageria	EmailJS	Plataforma <i>backend-as-a-service</i> para envio de e-mails
Hospedagem e Deploy	Vercel	Plataforma de Edge Network e hosting com deploy contínuo

5 Diagramas UML: Modelagem Visual do Sistema

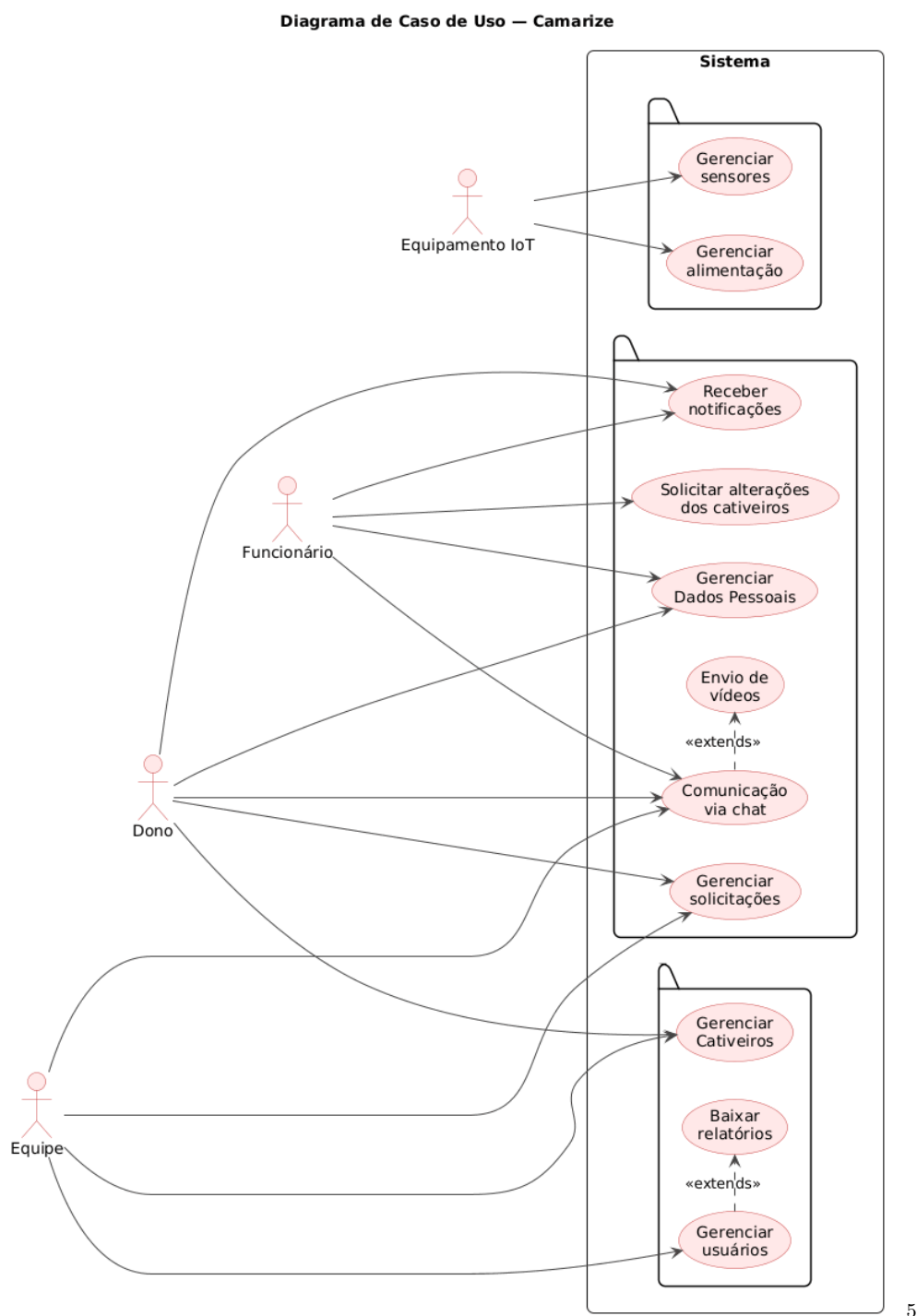
A Unified Modeling Language (UML) (BOOCH; RUMBAUGH; JACOBSON, 2005) é uma linguagem de modelagem visual usada para especificar, visualizar, construir e documentar artefatos de sistemas de software.⁴

⁴A UML foi padronizada pela Object Management Group (OMG) em 1997.

5.1 Diagrama de Casos de Uso

O diagrama de casos de uso representa as funcionalidades do sistema do ponto de vista do usuário:

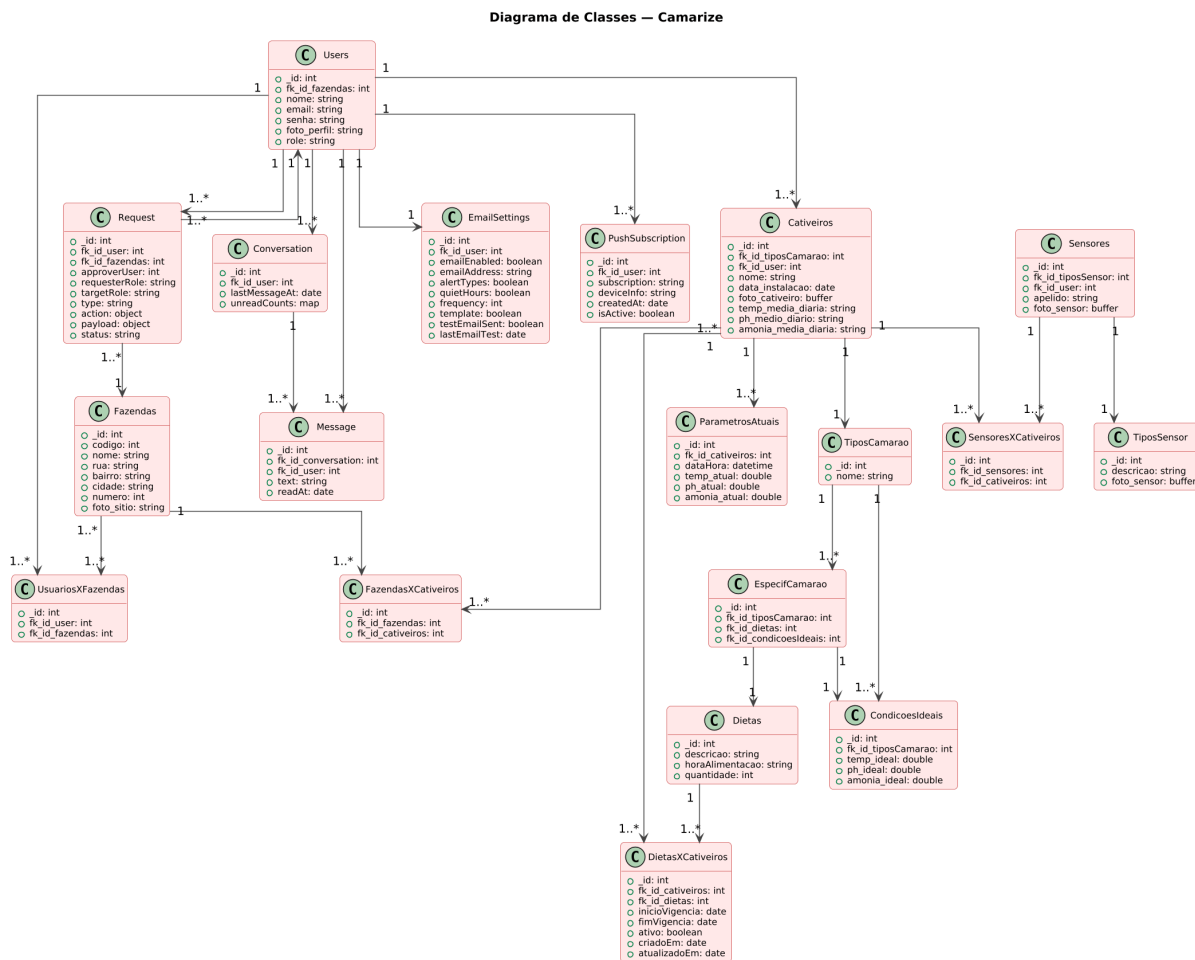
Figura 7: Diagrama de Casos de Uso



5.2 Diagrama de Classes

O diagrama de classes mostra a estrutura estática do sistema, incluindo todas as entidades, seus atributos, métodos e os relacionamentos entre elas.

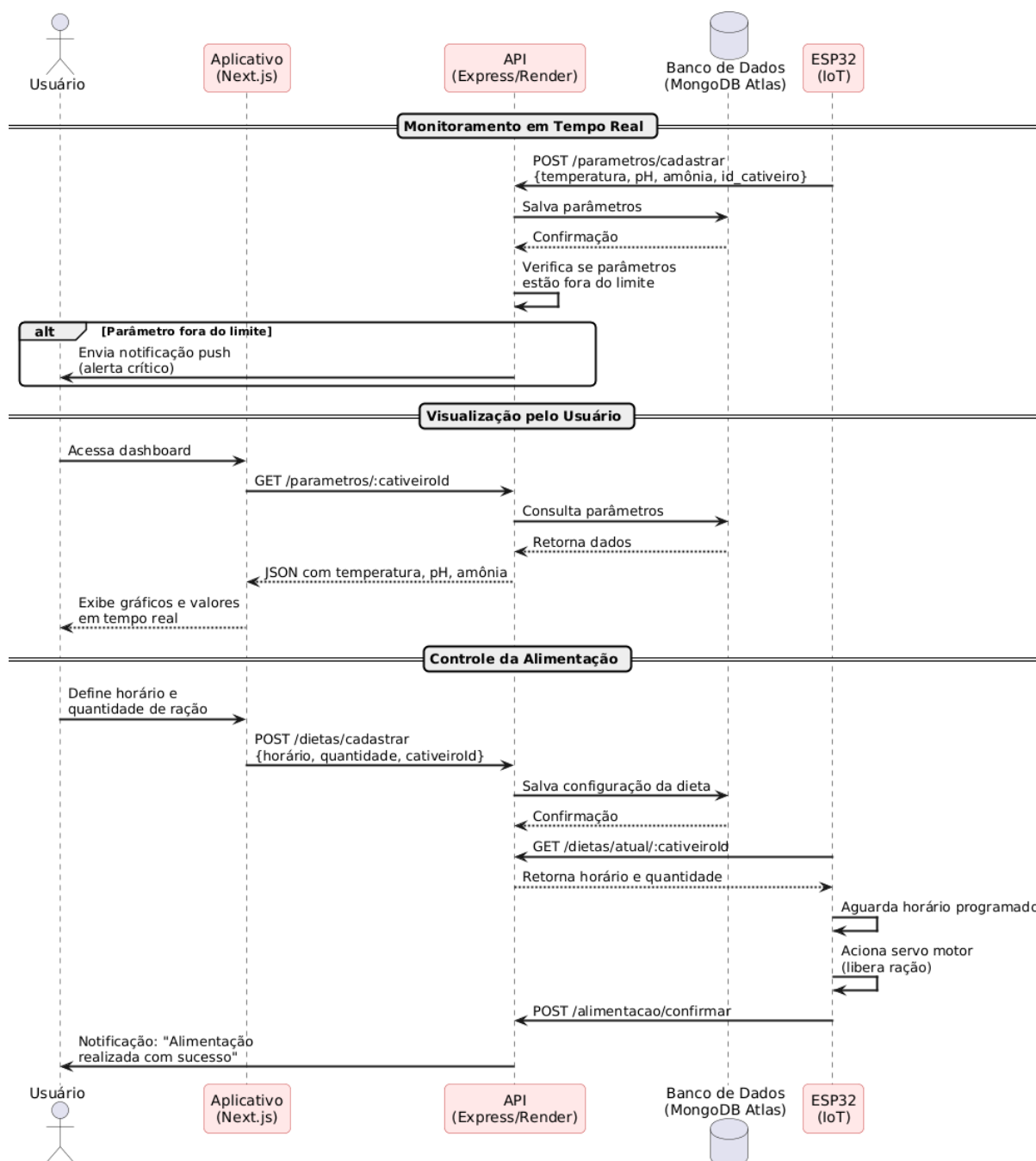
Figura 8: Diagrama de Classes



5.3 Diagrama de Sequência

O diagrama de sequência representa a interação entre os objetos ao longo do tempo, mostrando a ordem das mensagens trocadas para realizar uma funcionalidade específica.

Figura 9: Diagrama de Sequência

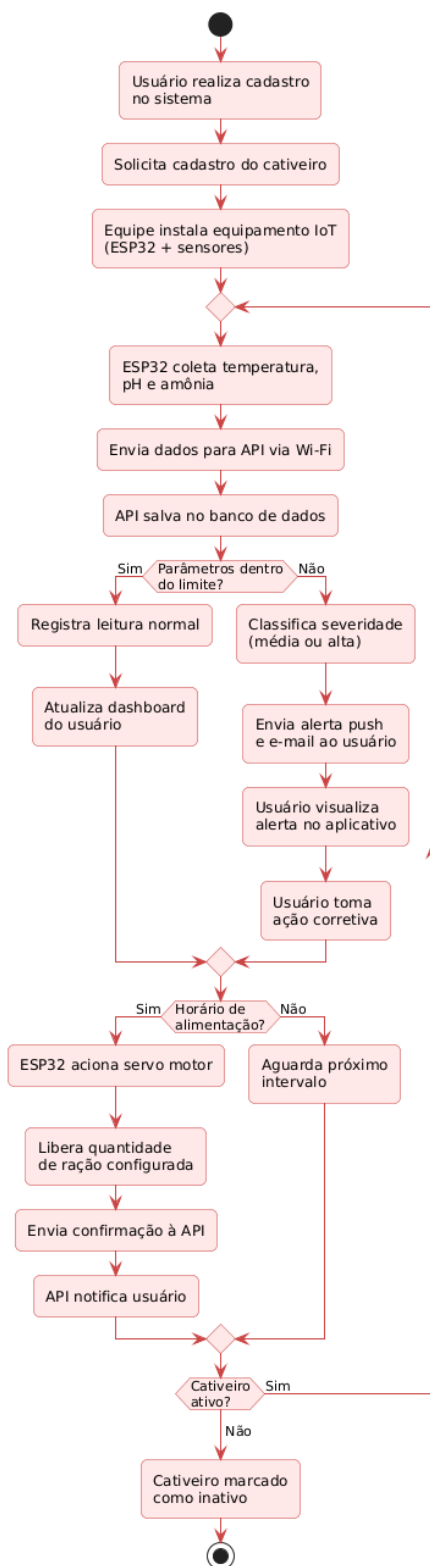


7

5.4 Diagrama de Atividades

O diagrama de atividades representa o fluxo de trabalho ou processo, mostrando as atividades, decisões e paralelismos envolvidos na execução de uma funcionalidade ou processo de negócio.

Figura 10: Diagrama de Atividades

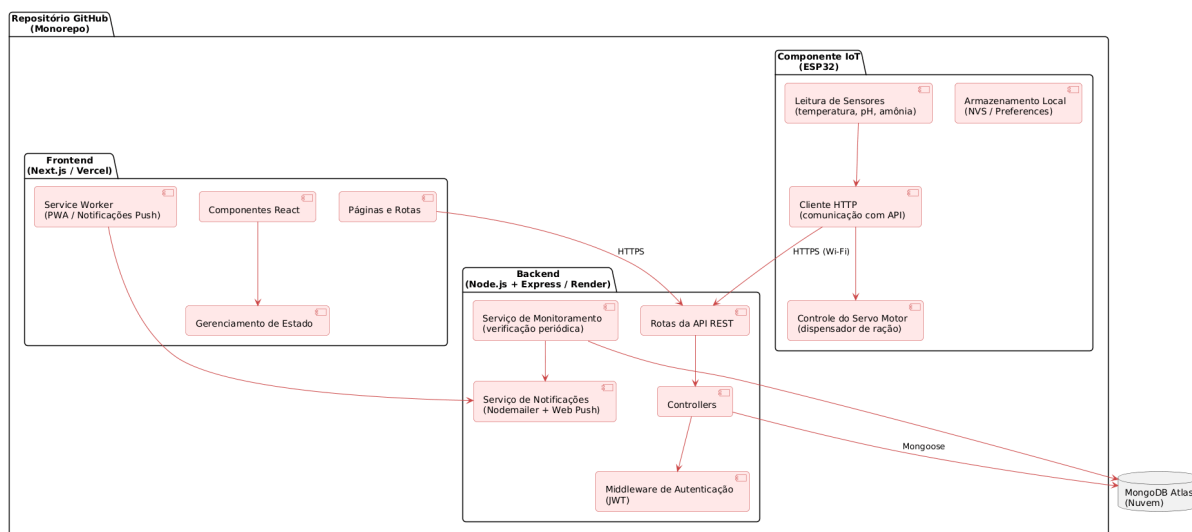


8

5.5 Diagrama de Componentes

O diagrama de componentes mostra a organização física dos componentes do sistema, incluindo módulos, bibliotecas e subsistemas, e como eles se relacionam.

Figura 11: Diagrama de Componentes

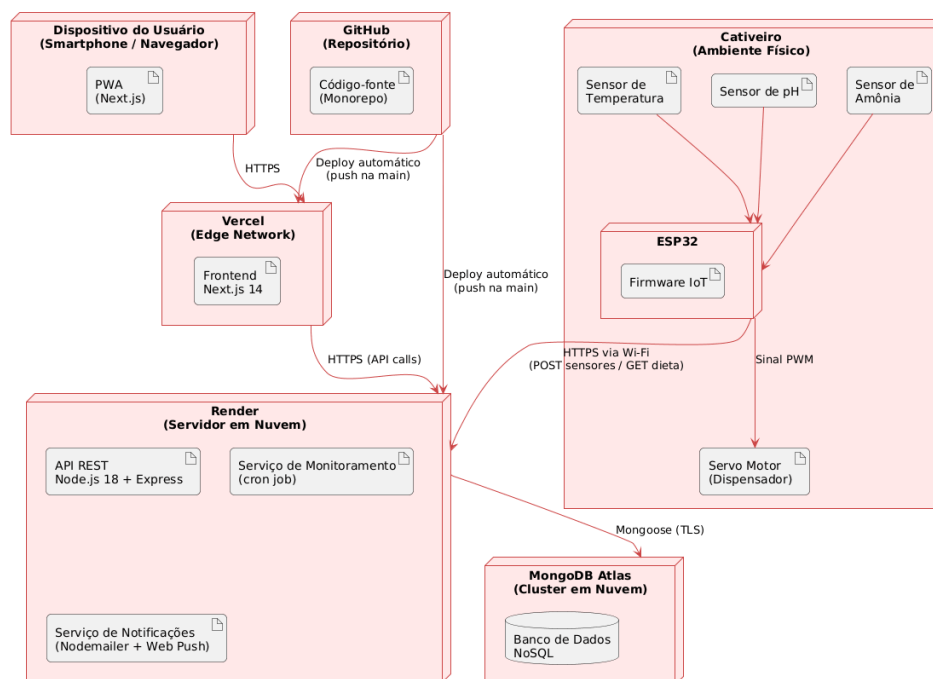


9

5.6 Diagrama de Implantação

O diagrama de implantação representa a arquitetura física do sistema, mostrando os nós (hardware) e os artefatos (software) implantados neles, bem como as conexões entre os nós.

Figura 12: Diagrama de Implantação

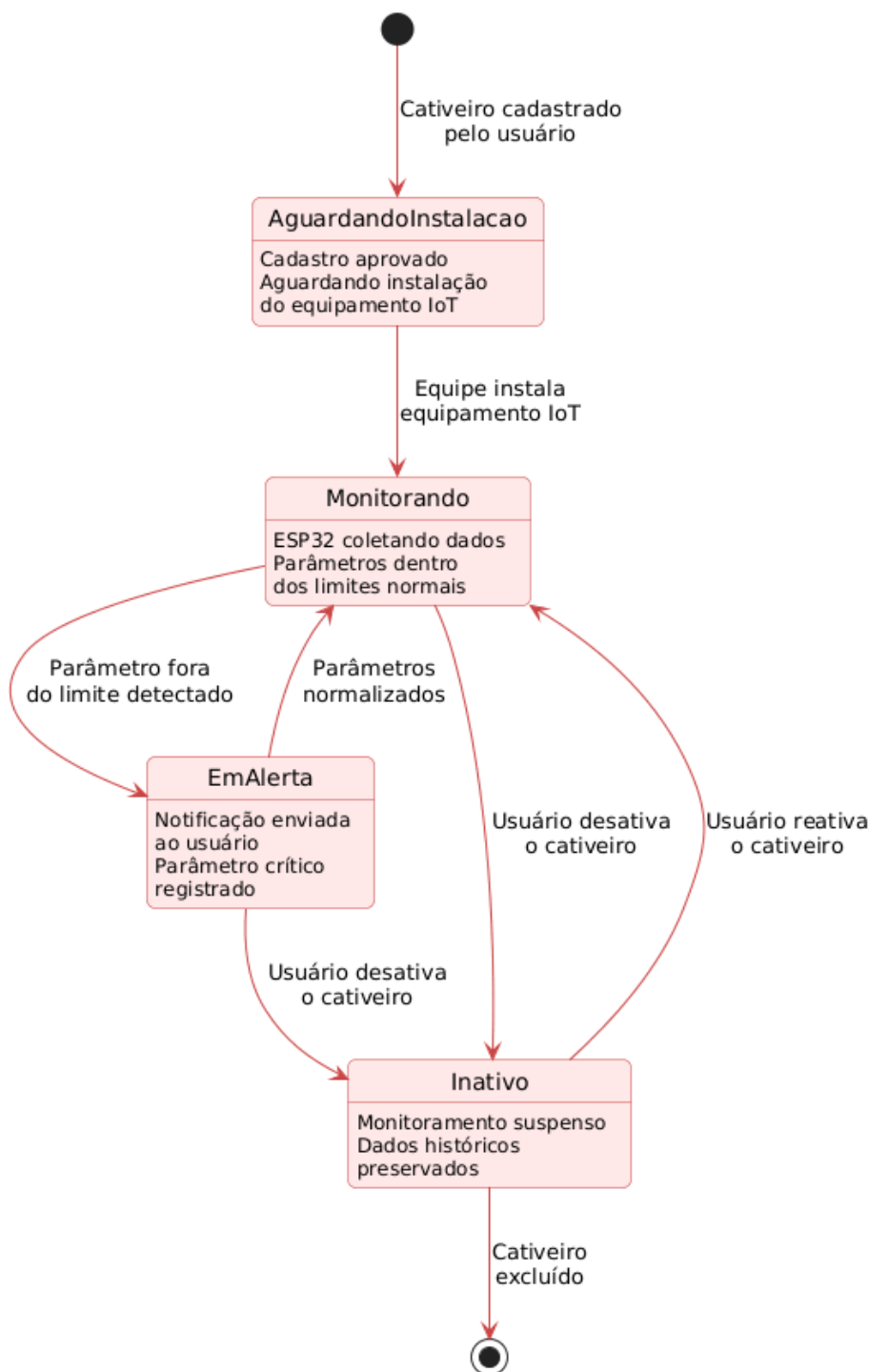


10

5.7 Diagrama de Estados

O diagrama de estados mostra os estados de um objeto e as transições entre esses estados, geralmente em resposta a eventos ou condições específicas.

Figura 13: Diagrama de Estados



11

Para realizar a montagem dos diagramas, foi utilizado a ferramenta online PlantUML, que permite a criação de diagramas UML através de código.¹²

¹²O PlantUML foi desenvolvido por Guillaume Boissinot e é amplamente utilizado em projetos de software para documentação e visualização de sistemas.

6 DevOps: Infraestrutura, Automação e Entrega Contínua

DevOps (KIM et al., 2018) é uma cultura e conjunto de práticas que integra desenvolvimento de software (Dev) e operações de TI (Ops), focando em automação, colaboração e entrega contínua.¹³

6.1 Pipeline CI/CD

O pipeline de integração contínua é executado automaticamente a cada push na branch `main`, garantindo a qualidade do código:

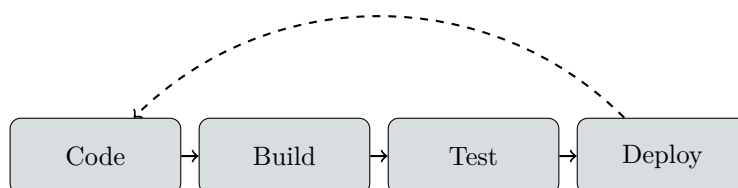


Figura 14: Pipeline CI/CD

6.2 Configuração CI/CD

O pipeline é composto por dois jobs paralelos: build do frontend (Next.js) e verificação de sintaxe da API (Node.js). Além disso, a Vercel realiza deploy automático do frontend a cada push:

```

name: CI
on:
  push:
    branches: [main]
jobs:
  build-frontend:
    name: Frontend build
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: front-react
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: npm
          cache-dependency-path: front-react/package-lock.json
      - run: npm ci
      - run: npm run build
  check-api:
    name: API syntax check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
  
```

¹³O termo "DevOps" foi popularizado por Patrick Debois em 2009.


```
with:
  node-version: 18
- run: find api -name "*.js" -not -path "*/node_modules/*"
  -exec node --check {} \;
```

6.3 Infraestrutura como Código

Tabela 5: Ferramentas DevOps Utilizadas

Categoria	Ferramenta	Finalidade
Controle de Versão	Git + GitHub	Versionamento de código e colaboração
CI/CD	GitHub Actions	Automação de build e verificação de sintaxe
Hosting Frontend	Vercel	Hospedagem e deploy automático do Next.js
Hosting Backend	Render	Hospedagem da API Express em produção
Containerização	Docker + Compose	Empacotamento e orquestração local de serviços
Banco de Dados	MongoDB Atlas	Banco de dados NoSQL em nuvem (cluster gerenciado)
Documentação API	Swagger UI	Documentação interativa de endpoints REST
Tunelamento	Ngrok	Exposição de serviços locais para testes externos

6.4 Infraestrutura do Sistema

Esta seção documenta como o sistema funciona em diferentes ambientes e plataformas.

6.4.1 Arquitetura de Rede

Tabela 6: Componentes de Infraestrutura

Componente	Descrição
Rede Local (LAN)	- Ambiente local para desenvolvimento e testes da aplicação - Docker Compose para integração entre API (porta 4000) e Frontend (porta 3000) - Comunicação entre serviços via <code>localhost</code> com portas dedicadas - Ngrok para exposição de serviços locais em túneis HTTP - Controle de versão integrado ao Git e GitHub - Testes de sintaxe e build executados via CI antes do merge
Cloud (Nuvem)	- Frontend hospedado na Vercel com deploy automático via integração GitHub - Backend hospedado no Render como serviço web - Banco de dados MongoDB Atlas em nuvem (cluster gerenciado) - Deploy automático do frontend a cada push na branch <code>main</code> - Replicação e backup automático fornecidos pelo MongoDB Atlas - Variáveis de ambiente gerenciadas nas plataformas Vercel e Render
Dispositivos Móveis	- Interface responsiva (PWA) adaptada para smartphones e tablets - Service Worker para cache offline e notificações push - Compatibilidade com navegadores móveis modernos (Chrome, Safari) - Push Notifications via Web Push API (VAPID) - Suporte para iOS e Android via navegador
Segurança	- SSL/TLS fornecido automaticamente pela Vercel e pelo Render - Autenticação via JSON Web Tokens (JWT) com validação de <code>tokenVersion</code> - Senhas criptografadas com <code>bcrypt</code> (migração transparente de senhas legadas) - CORS configurado para origens permitidas - Controle de acesso por roles (<code>master</code>, <code>admin</code>, <code>membro</code>) - Middleware de bloqueio de escrita para membros (<code>BlockMembersWrite</code>)

6.4.2 Diagrama de Infraestrutura

Para ilustrar como os componentes se conectam:

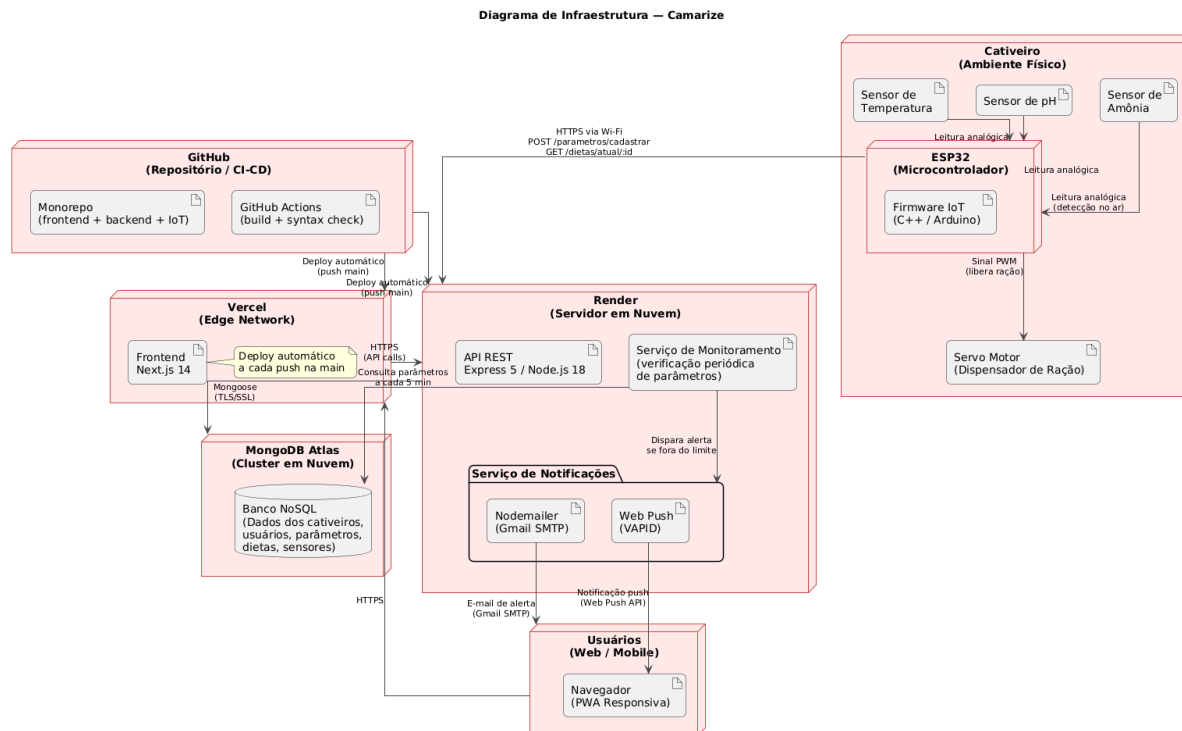


Figura 15: Diagrama de Infraestrutura do Sistema

- **Usuários Web/Mobile:** Acessam via navegador através de HTTPS (PWA responsiva)
- **Frontend:** Next.js 14 hospedado na Vercel (deploy automático)
- **Backend:** API Express 5 hospedada no Render (Node.js 18)
- **Banco de Dados:** MongoDB Atlas (cluster em nuvem)
- **Monitoramento:** Serviço interno com verificação periódica de parâmetros dos cativeiros
- **Notificações:** Nodemailer (Gmail SMTP) para alertas por e-mail + Web Push para notificações no navegador
- **ESP32:** Sensores IoT enviando dados de temperatura, pH e amônia via API REST

6.5 Arquitetura de Deploy

A arquitetura de deployment em produção utiliza serviços gerenciados na nuvem:

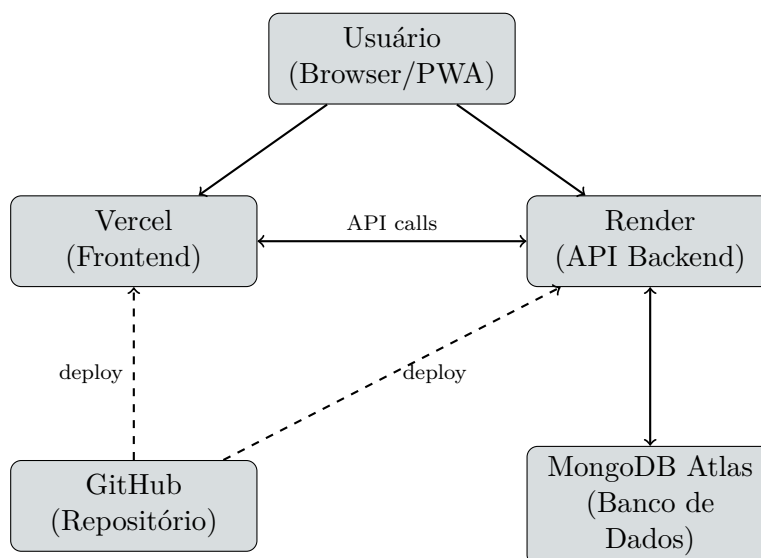


Figura 16: Arquitetura de Deployment em Produção

6.6 Configuração Docker Compose (Desenvolvimento)

Para o ambiente de desenvolvimento local, utiliza-se Docker Compose:

```

services:
  api:
    build: ../api
    container_name: camarize-api
    restart: unless-stopped
    env_file: ../api/.env
    environment:
      - NODE_ENV=production
      - PORT=4000
    ports:
      - "4000:4000"
  frontend:
    build: ../front-react
    container_name: camarize-frontend
    restart: unless-stopped
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:4000
      - NEXT_PUBLIC_SSE_URL=http://localhost:4000
    depends_on:
      - api
    ports:
      - "3000:3000"
  
```

6.7 Monitoramento e Alertas

O sistema possui um serviço de monitoramento automático que verifica periodicamente os parâmetros dos cativéis:

- **Intervalo de Verificação:** Configurável via variável de ambiente (MONITORING_INTERVAL_MINUTES, padrão: 5 minutos)

- **Parâmetros Monitorados:** Temperatura, pH e Amônia
- **Tolerâncias:** Temperatura (15%), pH (20%), Amônia (25%)
- **Severidade:** Classificação automática em *media* ou *alta*
- **Canais de Alerta:** E-mail (Gmail SMTP via Nodemailer) com respeito a quiet hours e rate limiting
- **Health Check:** Endpoint `/api/health` para verificação do status da API e banco de dados

6.8 Métricas e SLA

As métricas a seguir são **valores de referência** publicados pelas plataformas de hospedagem utilizadas (Vercel e Render), e não medições obtidas diretamente do projeto. Servem como parâmetros de expectativa para o comportamento da infraestrutura em produção:

- **Disponibilidade:** 99.9% (SLA garantido pela Vercel e Render em seus respectivos planos)
- **Tempo de Resposta:** < 200ms (média estimada, favorecida pela edge network da Vercel)
- **Taxa de Erro:** < 0.1% (referência da plataforma)
- **Throughput:** 1000 requisições/segundo (capacidade nominal da infraestrutura)
- **Deploy Frequency:** A cada push na branch `main` (deploy contínuo)
- **Lead Time:** < 1 hora (estimativa baseada no fluxo atual de desenvolvimento)
- **MTTR:** < 30 minutos (Mean Time To Recovery, estimativa operacional)

Conclusão

O conjunto de artefatos apresentado neste documento reflete o planejamento e a fundamentação técnica do projeto de monitoramento IoT para carcinicultura. A análise do Business Model Canvas evidenciou um modelo de negócio centrado na automação e na redução de custos operacionais para produtores de camarão, enquanto a análise SWOT identificou a dependência de conectividade e o custo inicial como principais desafios internos, contrabalançados pelo crescimento do mercado de aquicultura e pela demanda por soluções sustentáveis.

No âmbito técnico, os sete diagramas UML documentam a arquitetura completa do sistema, desde os casos de uso até a infraestrutura de implantação, fornecendo uma visão abrangente das interações entre os componentes. A infraestrutura DevOps implementada, com pipeline CI/CD via GitHub Actions, containerização com Docker e deploy contínuo nas plataformas Vercel e Render, garante um fluxo de entrega automatizado e rastreável.

O website institucional da equipe e o guia de estilos UX/UI complementam a documentação ao padronizar a identidade visual e os canais de comunicação do projeto. Em conjunto, os artefatos formam uma base documental coerente que sustenta o desenvolvimento, a manutenção e a evolução futura do sistema.

Referências

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *UML: Guia do Usuário*. 2. ed. Rio de Janeiro: Elsevier, 2005.

CHIAVENATO, I.; SAPIRO, A. *Planejamento Estratégico: da Intenção aos Resultados*. 4. ed. São Paulo: Atlas, 2020.

KIM, G. et al. *Manual de DevOps: Como Obter Agilidade, Confiabilidade e Segurança em Organizações Tecnológicas*. Rio de Janeiro: Alta Books, 2018.

OSTERWALDER, A.; PIGNEUR, Y. *Business Model Generation: Inovação em Modelos de Negócios*. Rio de Janeiro: Alta Books, 2011. Tradução da edição original em inglês de 2010.

ROGERS, Y.; SHARP, H.; PREECE, J. *Design de Interação: Além da Interação Humano-Computador*. 3. ed. Porto Alegre: Bookman, 2013.